



University
of Exeter

COMM113

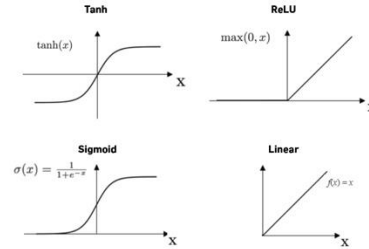
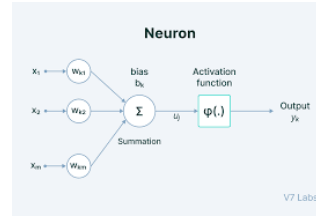
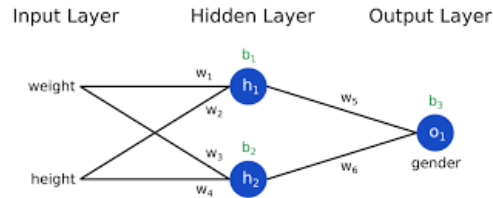
Deep Learning – Week 2

Neural Networks and Backpropagation

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

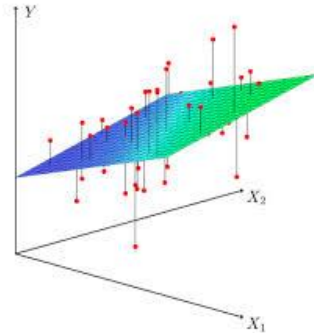
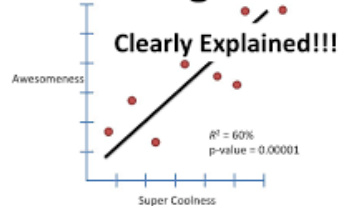
From Linear Models to Deep Learning

What you will learn: Neural network basics, structure, activation functions.



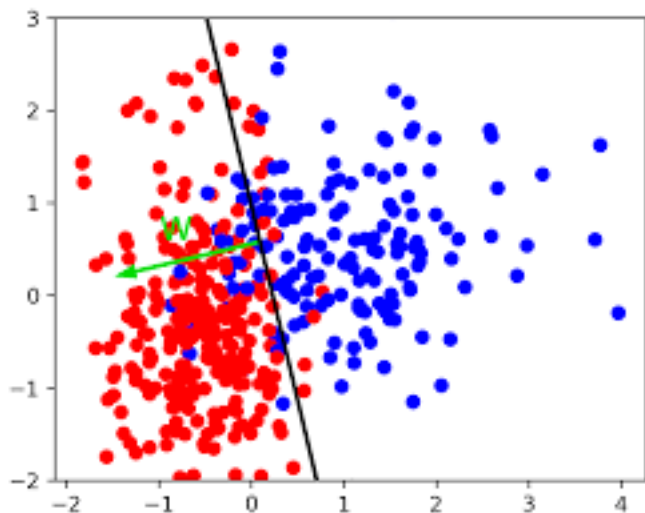
Motivation: Why not just linear models? (limitations in representing non-linear decision boundaries).

Linear Regression



What is the key idea that differentiates deep learning from linear models?

Limitation of Linear Models



They are “shallow” models, limited in expressivity.

They can only capture simple relationships. For example, if data is separated in a circle shape, a linear model cannot classify it well. Neural networks overcome this by combining many simple functions into more expressive ones.

Can you think of a dataset where linear models would fail?

Neural Network (Problem Formulation)

Vector function with tunable parameters θ

$$\mathbf{f}(\cdot; \theta) : \mathbb{R}^N \rightarrow (0, 1)^K$$

In a neural network, the input is a vector (like pixels in an image). The output can be in different forms:

Sample s in dataset S :

- input: $\mathbf{x}^s \in \mathbb{R}^N$
- expected output: $y^s \in [0, K - 1]$

1. Probability vector: $[0.2, 0.3, 0.5]$
2. One-hot vector: $[0, 0, 1]$
3. Label: $[2]$

Why do we often use probability vectors instead of just predicting a label?

Output is a conditional probability distribution:

$$\mathbf{f}(\mathbf{x}^s; \theta)_c = P(Y = c | X = \mathbf{x}^s)$$

Aim: maximise the conditional probability of True label

Neural Network (Problem Formulation)

Vector function with tunable parameters θ

$$\mathbf{f}(\cdot; \theta) : \mathbb{R}^N \rightarrow (0, 1)^K$$

Sample s in dataset S :

- input: $\mathbf{x}^s \in \mathbb{R}^N$
- expected output: $y^s \in [0, K - 1]$

1. Probability vector: $[0.2, 0.3, 0.5]$
2. One-hot vector: $[0, 0, 1]$
3. Label: $[2]$

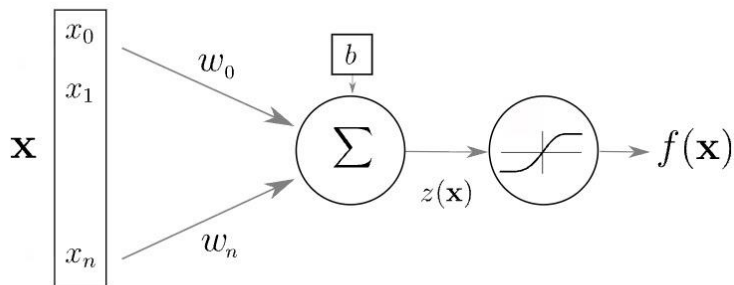
Why do we often use probability vectors instead of just predicting a label?

capture uncertainty and are useful for decision-making

Output is a conditional probability distribution:

$$\mathbf{f}(\mathbf{x}^s; \theta)_c = P(Y = c | X = \mathbf{x}^s)$$

Neuron



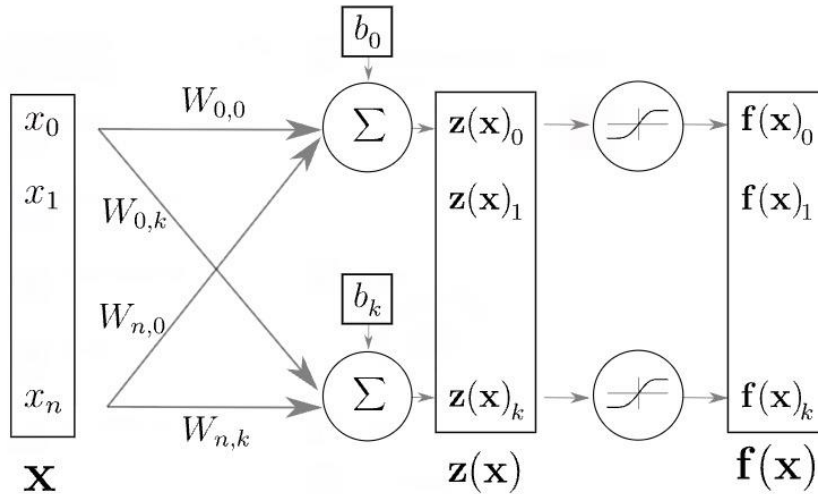
$$z(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

$$f(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x} + b)$$

- $\mathbf{x}, f(\mathbf{x})$ input and output
- $z(\mathbf{x})$ pre-activation
- $\mathbf{w}, b$ weights and bias
- g activation function

This is the fundamental unit: the neuron. It takes weighted inputs, applies a bias, and passes the result through an activation function.

Layer of Neurons



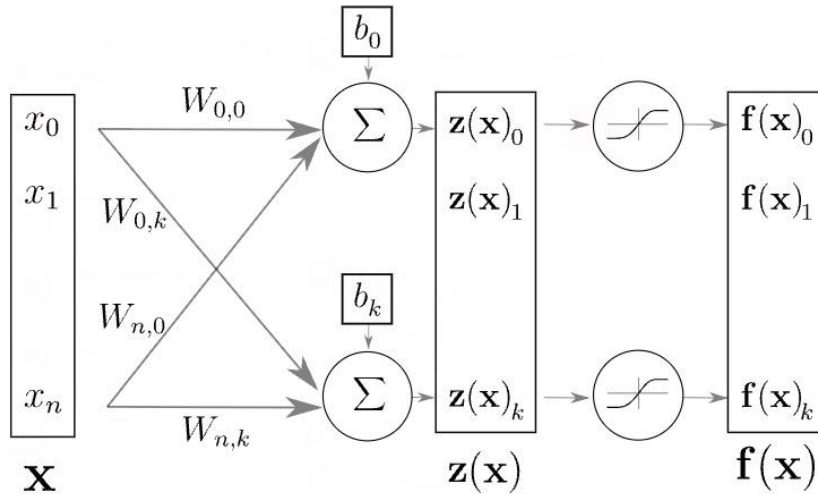
Neurons are arranged in layers. Each layer transforms its input into a new representation. Stacking layers creates a hierarchy of feature extraction — from simple to complex.

In an image classifier, what might the first hidden layer learn?

$$\mathbf{f}(\mathbf{x}) = \mathbf{g}(\mathbf{z}(\mathbf{x})) = \mathbf{g}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- $\mathbf{W}, \mathbf{b}$ now matrix and vector

Layer of Neurons



$$\mathbf{f}(\mathbf{x}) = \mathbf{g}(\mathbf{z}(\mathbf{x})) = \mathbf{g}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- $\mathbf{W}, \mathbf{b}$ now matrix and vector

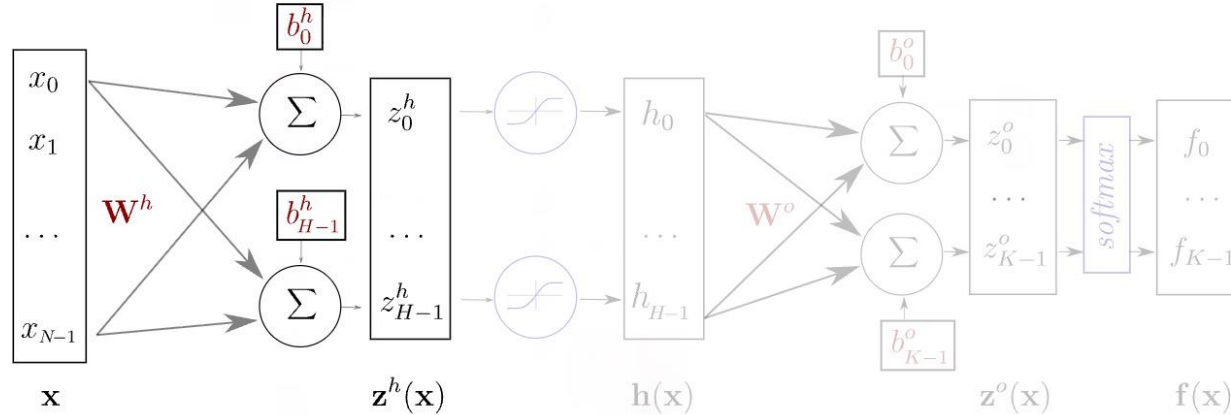
Neurons are arranged in layers. Each layer transforms its input into a new representation. Stacking layers creates a hierarchy of feature extraction — from simple to complex.

In an image classifier, what might the first hidden layer learn?

Simple features like edges or colours.

One Hidden Layer Network

Can also be called: Fully connected neural network or Multilayer perceptron

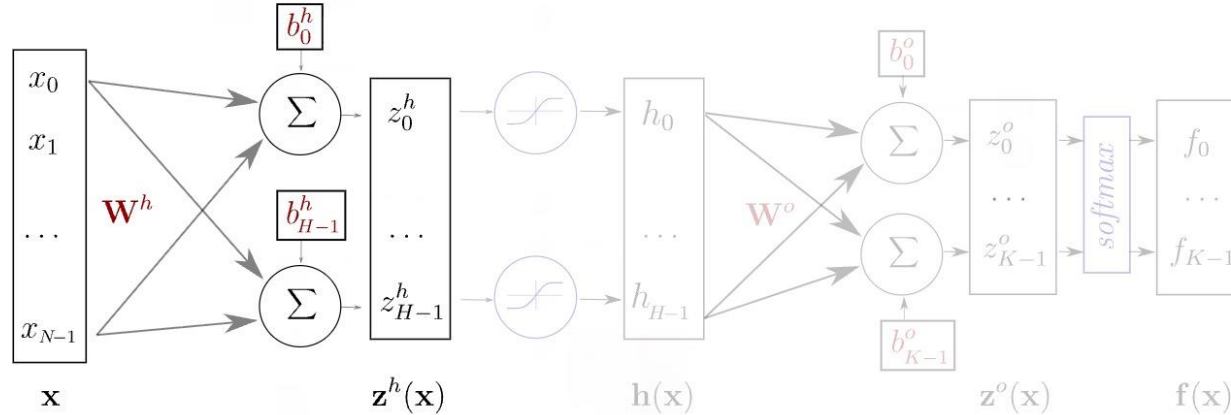


Q: Why is this called a “fully connected” network?

- $\mathbf{z}^h(\mathbf{x}) = \mathbf{W}^h \mathbf{x} + \mathbf{b}^h$
- $\mathbf{h}(\mathbf{x}) = g(\mathbf{z}^h(\mathbf{x})) = g(\mathbf{W}^h \mathbf{x} + \mathbf{b}^h)$
- $\mathbf{z}^o(\mathbf{x}) = \mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o$
- $\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o) = \text{softmax}(\mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o)$

One Hidden Layer Network

Can also be called: Fully connected neural network or Multilayer perceptron

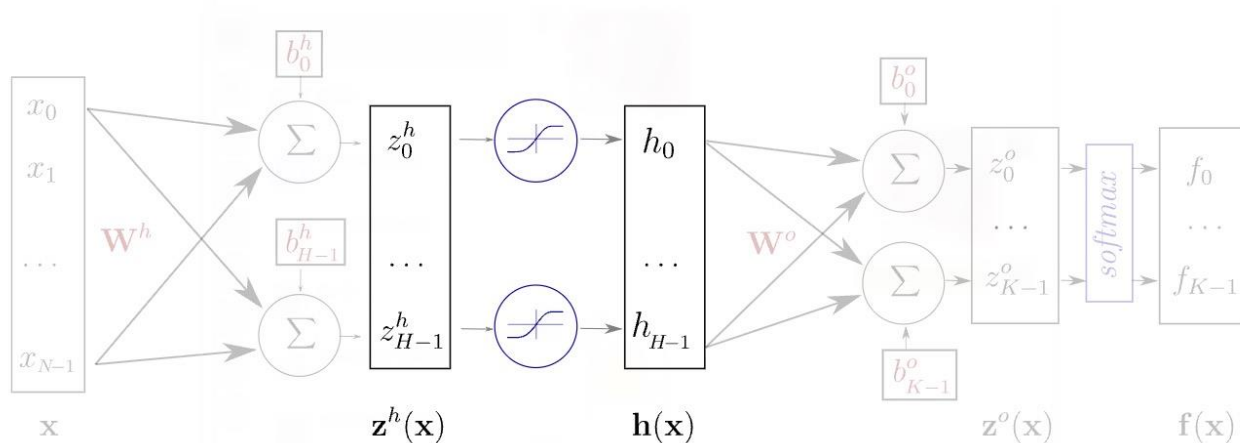


Q: Why is this called a “fully connected” network?

A: Because every input node connects to every hidden node, and every hidden node to the output.

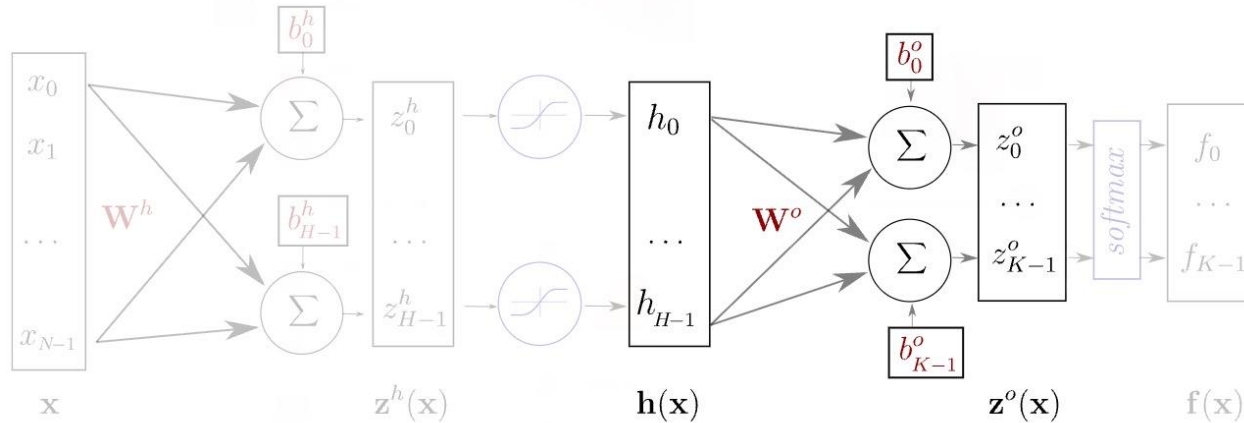
- $\mathbf{z}^h(\mathbf{x}) = \mathbf{W}^h \mathbf{x} + \mathbf{b}^h$
- $\mathbf{h}(\mathbf{x}) = g(\mathbf{z}^h(\mathbf{x})) = g(\mathbf{W}^h \mathbf{x} + \mathbf{b}^h)$
- $\mathbf{z}^o(\mathbf{x}) = \mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o$
- $\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o) = \text{softmax}(\mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o)$

One Hidden Layer Network



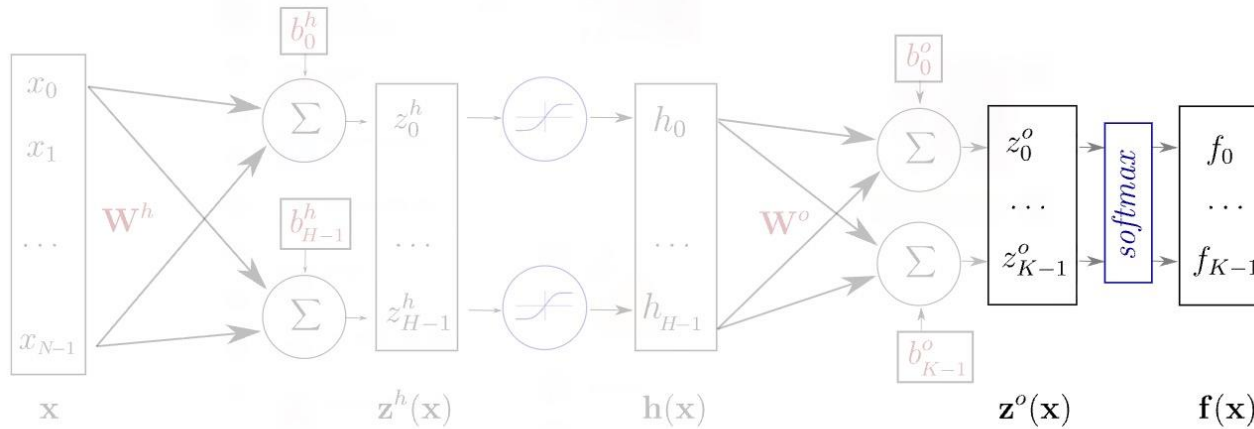
- $\mathbf{z}^h(\mathbf{x}) = \mathbf{W}^h \mathbf{x} + \mathbf{b}^h$
- $\mathbf{h}(\mathbf{x}) = g(\mathbf{z}^h(\mathbf{x})) = g(\mathbf{W}^h \mathbf{x} + \mathbf{b}^h)$
- $\mathbf{z}^o(\mathbf{x}) = \mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o$
- $\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o) = \text{softmax}(\mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o)$

One Hidden Layer Network



- $\mathbf{z}^h(\mathbf{x}) = \mathbf{W}^h \mathbf{x} + \mathbf{b}^h$
- $\mathbf{h}(\mathbf{x}) = g(\mathbf{z}^h(\mathbf{x})) = g(\mathbf{W}^h \mathbf{x} + \mathbf{b}^h)$
- $\mathbf{z}^o(\mathbf{x}) = \mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o$
- $\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o) = \text{softmax}(\mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o)$

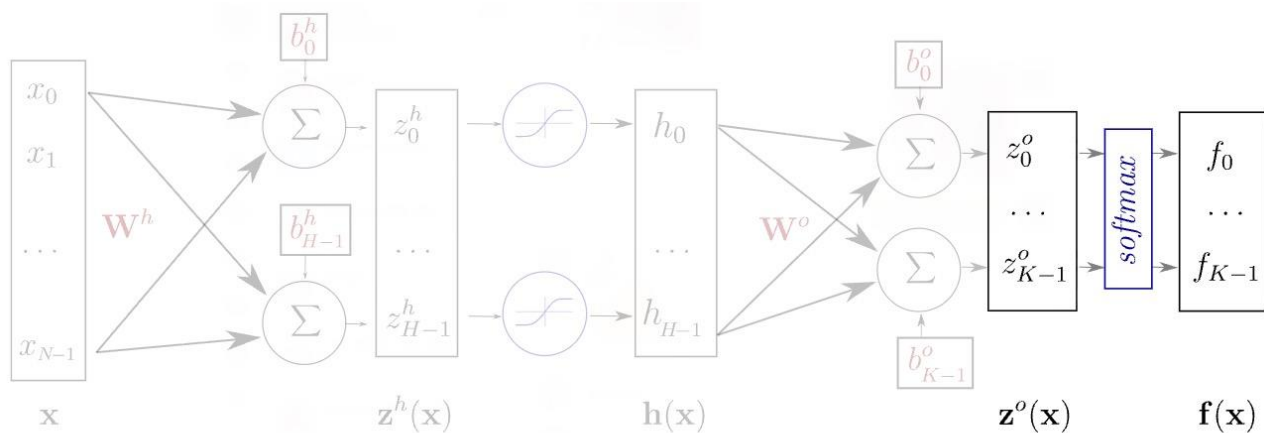
One Hidden Layer Network



Q: How can adding more hidden neurons affect the model?

- $\mathbf{z}^h(\mathbf{x}) = \mathbf{W}^h \mathbf{x} + \mathbf{b}^h$
- $\mathbf{h}(\mathbf{x}) = g(\mathbf{z}^h(\mathbf{x})) = g(\mathbf{W}^h \mathbf{x} + \mathbf{b}^h)$
- $\mathbf{z}^o(\mathbf{x}) = \mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o$
- $\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o) = \text{softmax}(\mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o)$

One Hidden Layer Network

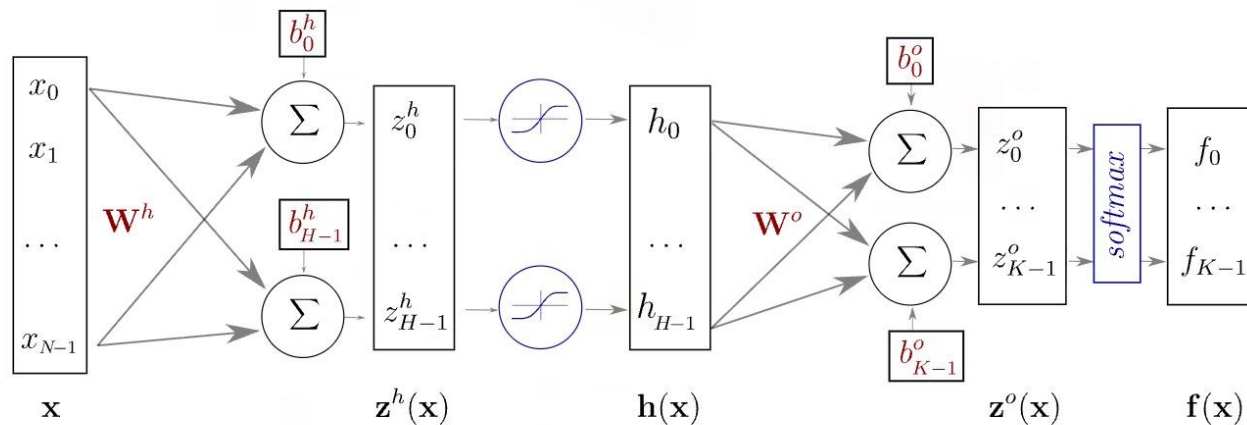


- $\mathbf{z}^h(\mathbf{x}) = \mathbf{W}^h \mathbf{x} + \mathbf{b}^h$
- $\mathbf{h}(\mathbf{x}) = g(\mathbf{z}^h(\mathbf{x})) = g(\mathbf{W}^h \mathbf{x} + \mathbf{b}^h)$
- $\mathbf{z}^o(\mathbf{x}) = \mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o$
- $\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o) = \text{softmax}(\mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o)$

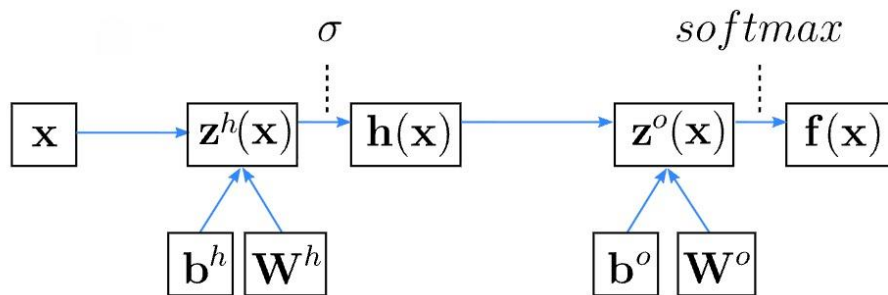
Q: How can adding more hidden neurons affect the model?

A: It increases capacity but may risk overfitting.

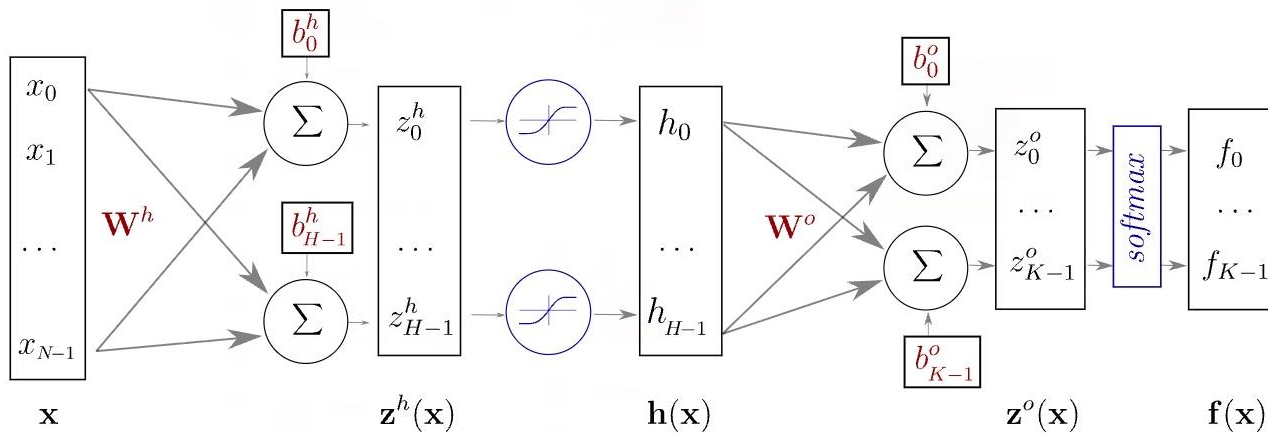
One Hidden Layer Network



Alternate representation



One Hidden Layer Network

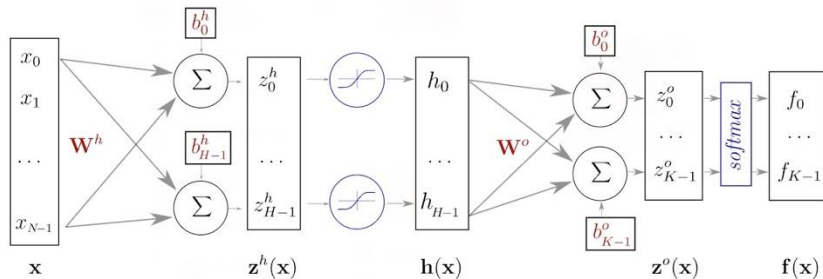


PyTorch Implementation

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(N, H),          # weight matrix dim [N * H]
    nn.Tanh(),
    nn.Linear(H, K),          # weight matrix dim [H x K]
    nn.Softmax(dim=1)
)
```

One Hidden Layer Network



PyTorch Implementation 2

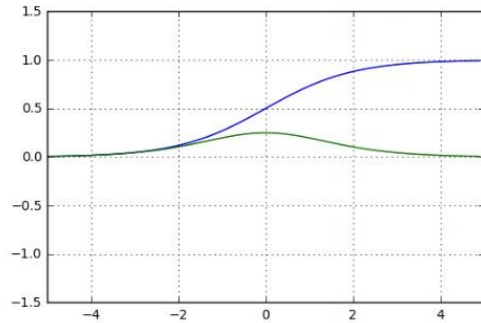
```
import torch
import torch.nn as nn
```

```
class OneHiddenLayerNetwork(nn.Module):
    def __init__(self, N, H, K):
        super(OneHiddenLayerNetwork, self).__init__()
        self.fc1 = nn.Linear(N, H)
        self.tanh = nn.Tanh()
        self.fc2 = nn.Linear(H, K)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.tanh(x)
        x = self.fc2(x)
        x = self.softmax(x)
        return x
```

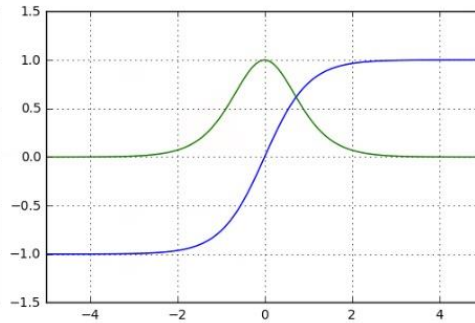
```
model = OneHiddenLayerNetwork(N, H, K)
```

Activation Functions



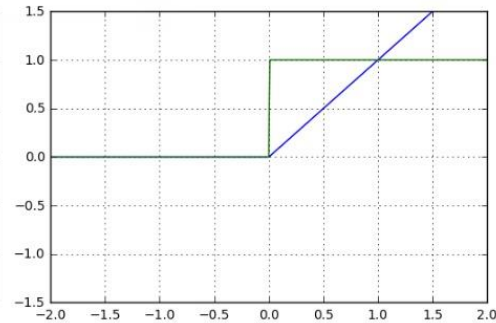
$$\text{sigm}(x) = \frac{1}{1 + e^{-x}}$$

$$\text{sigm}'(x) = \text{sigm}(x)(1 - \text{sigm}(x))$$



$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

$$\tanh'(x) = 1 - \tanh(x)^2$$



$$\text{relu}(x) = \max(0, x)$$

$$\text{relu}'(x) = 1_{x>0}$$

- blue: activation function
- green: derivative

Enhance expression ability

Softmax

$$\text{softmax}(\mathbf{x}) = \frac{1}{\sum_{i=1}^n e^{x_i}} \cdot \begin{bmatrix} e^{x_1} \\ e^{x_2} \\ \vdots \\ e^{x_n} \end{bmatrix}$$

Q: Why can't we just use the largest logit directly as the prediction?

$$\frac{\partial \text{softmax}(\mathbf{x})_i}{\partial x_j} = \begin{cases} \text{softmax}(\mathbf{x})_i \cdot (1 - \text{softmax}(\mathbf{x})_i) & i = j \\ -\text{softmax}(\mathbf{x})_i \cdot \text{softmax}(\mathbf{x})_j & i \neq j \end{cases}$$

A softmax layer in a neural network takes raw scores (logits) from the previous layer and **transforms them into a probability distribution**, where each output value is between 0 and 1 and the sum of all values equals 1.

Softmax

$$\text{softmax}(\mathbf{x}) = \frac{1}{\sum_{i=1}^n e^{x_i}} \cdot \begin{bmatrix} e^{x_1} \\ e^{x_2} \\ \vdots \\ e^{x_n} \end{bmatrix}$$

Q: Why can't we just use the largest logit directly as the prediction?

A: We could, but probabilities are more interpretable and useful for training with cross-entropy loss.

$$\frac{\partial \text{softmax}(\mathbf{x})_i}{\partial x_j} = \begin{cases} \text{softmax}(\mathbf{x})_i \cdot (1 - \text{softmax}(\mathbf{x})_i) & i = j \\ -\text{softmax}(\mathbf{x})_i \cdot \text{softmax}(\mathbf{x})_j & i \neq j \end{cases}$$

A softmax layer in a neural network takes raw scores (logits) from the previous layer and **transforms them into a probability distribution**, where each output value is between 0 and 1 and the sum of all values equals 1.

Practice

```
X = torch.tensor([
    [1.0, 2.0, 3.0, 4.0],
    [2.0, 3.0, 4.0, 5.0],
    [3.0, 4.0, 5.0, 6.0],
    [4.0, 5.0, 6.0, 7.0],
    [5.0, 6.0, 7.0, 8.0]
], dtype=torch.float32)
```

Input set: 5 data, each data is a vector with 4 features

Label set: A vector, each element represents the true label of corresponding input data

```
y = torch.tensor([0, 1, 2, 1, 0], dtype=torch.long)
```